Глубокое обучение с подкреплением

Бонусный модуль 1: Обучаем Huggy и играем с ним

Курс Hugging Face Deep RL
Полная учебная версия на основе MDX

Источник: курс Hugging Face Deep RL (оригинальный английский текст).

Содержание

1 Введение **2**

2 Как устроен Huggy **2**
2.1 Пространство состояний: что воспринимает Huggy . . . 3
2.2 Пространство действий: как может двигаться Huggy . . . 3
2.3 Функция вознаграждения . . . 3
2.4 Обучение Huggy . . . 4

3 Обучаем Huggy и играем с ним **5**
3.1 Приступим к обучению Huggy . . . 5
3.2 Цели практического ноутбука . . . 6
3.3 Предварительные требования . . . 7
3.4 Настройка GPU . . . 7
3.5 Клонирование репозитория . . . 8
3.6 Настройка виртуального окружения . . . 8
3.7 Установка зависимостей . . . 9
3.8 Загрузка и распаковка архива среды . . . 9
3.9 Повторим устройство среды . . . 9
3.10 Проверка конфигурации Huggy . . . 11
3.11 Обучение агента . . . 12
3.12 Публикация агента на Hugging Face Hub . . . 13
3.13 Играем со своим Huggy . . . 15
3.14 Продолжайте учиться и открывать новое . . . 16

4 Игра с Huggy **16**

5 Заключение **17**

1 Введение

В этом бонусном модуле мы закрепим материал первого модуля: научим собаку Huggy подбирать брошенную палку, а затем сможем поиграть с обученным агентом прямо в браузере.



Рис. 1: Huggy: обложка бонусного модуля курса Hugging Face [оригинал]

Итак, приступим!

2 Как устроен Huggy

Huggy — среда для глубокого обучения с подкреплением, созданная Hugging Face на основе Puppo the Corgi — проекта команды Unity ML-Agents. Среда разработана с помощью игрового движка Unity и ML-Agents. ML-Agents — набор инструментов для Unity, позволяющий **создавать собственные среды или использовать готовые среды для обучения агентов**.



Рис. 2: Huggy: собака-агент в среде Unity ML-Agents [оригинал]

В этой среде мы хотим научить Huggy **подбирать брошенную нами палку. Для этого ему необходимо правильно перемещаться в её направлении**.

2.1 Пространство состояний: что воспринимает Huggy

Huggy не «видит» окружающую среду. Вместо этого мы предоставляем ему следующую информацию:

- Положение цели (палки)
- Относительное положение Huggy и цели
- Ориентация его лап.

На основе этой информации Huggy может **использовать свою политику, чтобы выбрать следующее действие для достижения цели**.

2.2 Пространство действий: как может двигаться Huggy

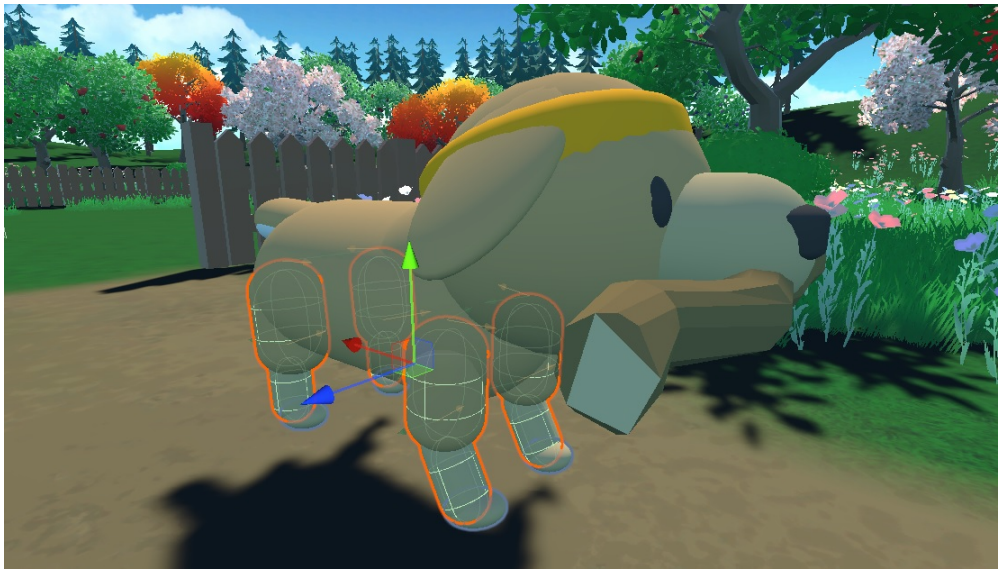


Рис. 3: Управление суставными двигателями лап Huggy [оригинал]

Лапы Huggy приводятся в движение суставными двигателями. Следовательно, чтобы добраться до цели, Huggy должен **научиться правильно вращать двигатели суставов каждой лапы и таким образом перемещаться**.

2.3 Функция вознаграждения

Функция вознаграждения спроектирована так, чтобы **Huggy достигал своей цели**: находил палку.

Напомним, что одно из фундаментальных положений обучения с подкреплением — *гипотеза о вознаграждении*: цель можно описать как **максимизацию ожидаемого суммарного вознаграждения**.

В нашем случае Huggy должен **приближаться к палке, не вращаясь слишком сильно**. Поэтому функция вознаграждения должна отражать именно эту цель.

Наша функция вознаграждения:



Рис. 4: Функция вознаграждения для обучения Huggy [оригинал]

- *Бонус за ориентацию*: мы **поощряем Huggy за приближение к цели**.
- *Штраф за время*: постоянный штраф на каждом шаге, который **побуждает его добраться до палки как можно быстрее**.
- *Штраф за вращение*: мы штрафуем Huggy, если **он слишком сильно вращается и слишком резко поворачивает**.
- *Вознаграждение за достижение цели*: поощряем Huggy за то, что **он достиг цели**.

Если вы хотите увидеть математическое описание этой функции вознаграждения, посмотрите презентацию Puppo the Corgi.

2.4 Обучение Huggy

Задача Huggy — **научиться правильно и как можно быстрее бежать к цели**. Для этого на каждом шаге, получив наблюдение среды, он должен решить, как вращать суставные двигатели лап, чтобы двигаться правильно (без избыточного вращения) и в направлении цели.

Цикл обучения выглядит следующим образом:

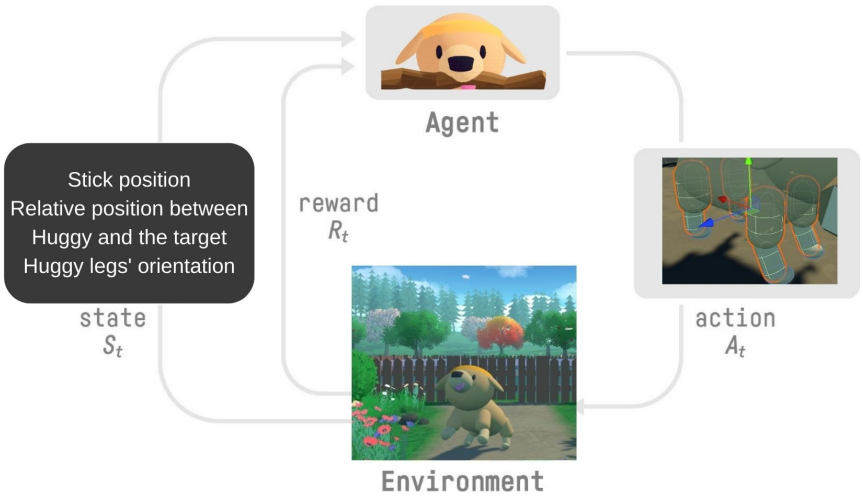


Рис. 5: Цикл взаимодействия агента Huggy со средой и его обучения [оригинал]

Среда обучения выглядит так:

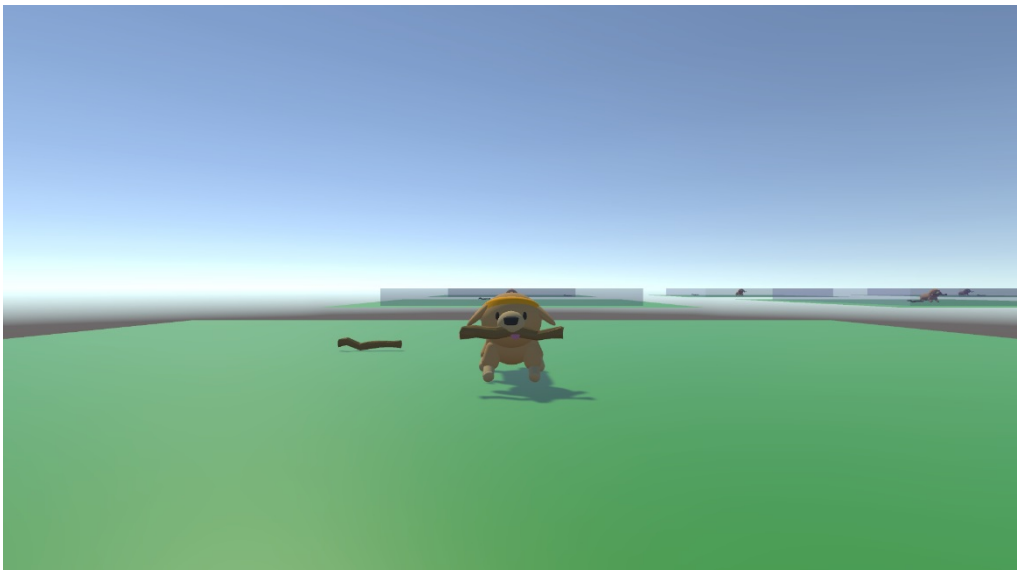


Рис. 6: Несколько параллельных тренировочных арен со случайным положением палки [оригинал]

Здесь **палка появляется в случайной точке**. Когда Huggy достигает её, палка возникает в другом месте. Для обучения мы создали **несколько копий среды**. Это ускоряет обучение, поскольку агент получает более разнообразный опыт.

Теперь, когда у вас есть общее представление о среде, вы готовы обучить Huggy подбирать палку.

Для этого мы воспользуемся ML-Agents. Не переживайте, если раньше не работали с этим набором инструментов. В данном модуле мы обучим Huggy в Google Colab, а затем вы сможете загрузить своего обученного агента и поиграть с ним прямо в браузере.

В одном из следующих модулей мы подробнее изучим ML-Agents и принципы его работы. А пока упростим задачу и воспользуемся готовой реализацией.

3 Обучаем Huggy и играем с ним

Открыть ноутбук с обучением Huggy в Google Colab

Мы настоятельно **рекомендуем студентам выполнять практические задания в Google Colab**, а не на личных компьютерах.

Используя Google Colab, **можно сосредоточиться на обучении и экспериментах, не отвлекаясь на технические вопросы** настройки окружения.

3.1 Приступим к обучению Huggy

Чтобы начать обучение Huggy, нажмите кнопку Open in Colab:

Открыть в Google Colab



Рис. 7: Huggy: обложка бонусного модуля курса Hugging Face [оригинал]

В этом ноутбуке мы закрепим материал первого модуля: **научим Huggy подбирать палку и затем поиграем с обученным агентом прямо в браузере**.



Рис. 8: Huggy: собака-агент в среде Unity ML-Agents [оригинал]

Среда

- Huggy — собака-агент в среде, созданной Томасом Симонини на основе проекта Puppo the Corgi.

Используемая библиотека

- MLAgents

Мы постоянно стараемся улучшать наши руководства, поэтому **если вы обнаружили проблему в этом ноутбуке**, пожалуйста, создайте issue в репозитории GitHub.

3.2 Цели практического ноутбука

По окончании работы с ноутбуком вы сможете:

- Разобраться в **пространстве состояний, пространстве действий и функции вознаграждения**, используемых для обучения Huggy.

- **Обучить собственного Huggy** подбирать палку.
- **Играть с обученным Huggy прямо в браузере**.

3.3 Предварительные требования

Прежде чем приступить к ноутбуку, необходимо:

[ ] **Разобраться с основами обучения с подкреплением** (MC, TD, гипотеза о вознаграждении и т. д.), пройдя первый модуль.

[ ] **Прочитать введение в работу Huggy** в бонусном модуле 1.

3.4 Настройка GPU

- Для **ускорения обучения агента мы будем использовать GPU**. Перейдите в `Runtime > Change Runtime type`.

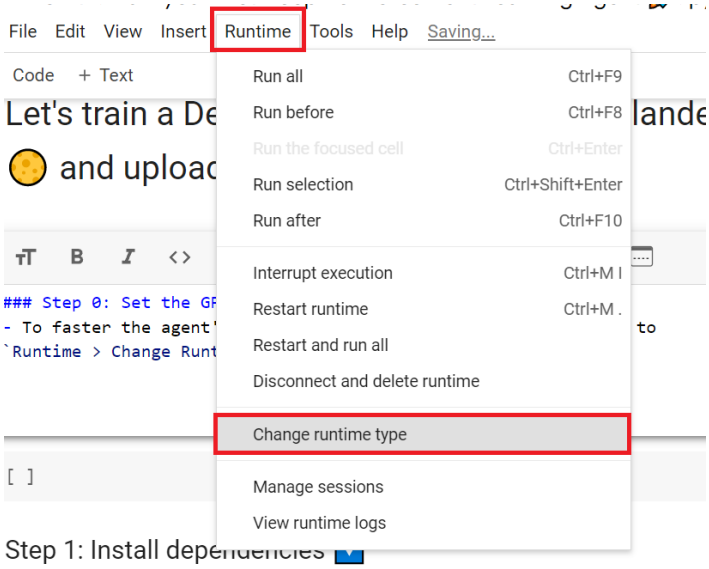


Рис. 9: Google Colab: изменение типа среды выполнения [оригинал]

- Выберите `Hardware Accelerator > GPU`.

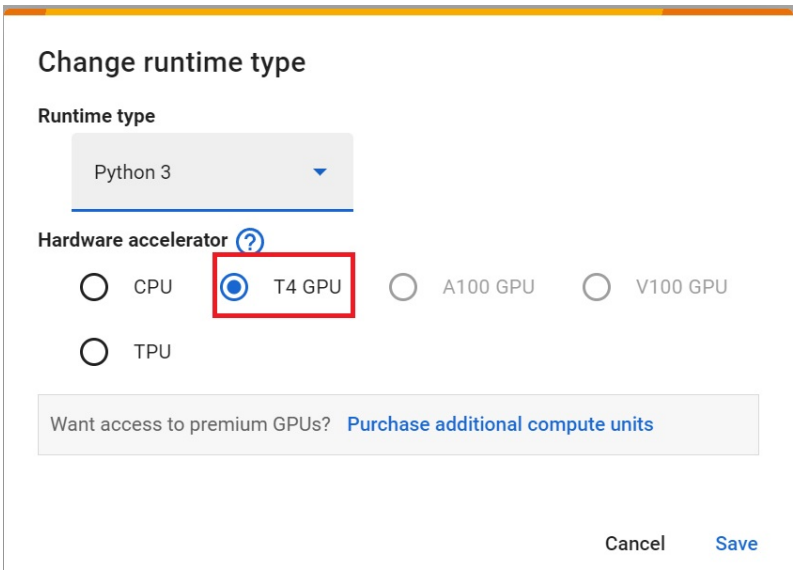


Рис. 10: Google Colab: выбор GPU как аппаратного ускорителя [оригинал]

3.5 Клонирование репозитория

- Нужно клонировать репозиторий, содержащий **ML-Agents**.

```
# Clone the repository (can take 3min)
git clone --depth 1 https://github.com/Unity-Technologies/ml-agents
```

3.6 Настройка виртуального окружения

- Для корректной работы **ML-Agents** в Colab версия Python должна соответствовать требованиям этой библиотеки.
- Поддерживаемую версию Python можно узнать по параметру `python_requires` в файлах `setup.py`. Эти файлы нужны для установки **ML-Agents**; они расположены по следующим путям:
  - `/content/ml-agents/ml-agents/setup.py`
  - `/content/ml-agents/ml-agents-envs/setup.py`
- Текущая версия Python в Colab (её можно проверить с помощью `!python --version`) не соответствует параметру `python_requires` библиотеки. Из-за этого установка может завершиться без явной ошибки, однако при последующем выполнении команд возникнут такие сообщения:
  - `/bin/bash: line 1: mlagents-learn: command not found`
  - `/bin/bash: line 1: mlagents-push-to-hf: command not found`
- Для решения проблемы создадим виртуальное окружение с версией Python, совместимой с **ML-Agents**.

`Примечание:` *Для совместимости с будущими версиями всегда проверяйте параметр* ***`python_requires`*** *в установочных файлах. Если версия Python в Colab не поддерживается, укажите в приведённом ниже скрипте максимально поддерживаемую библиотекой версию Python.*

```
# Colab's Current Python Version (Incompatible with ML-Agents)
!python --version
```

```
# Install virtualenv and create a virtual environment
!pip install virtualenv
!virtualenv myenv

# Download and install Miniconda
!wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
!chmod +x Miniconda3-latest-Linux-x86_64.sh
!./Miniconda3-latest-Linux-x86_64.sh -b -f -p /usr/local

# Activate Miniconda and install Python ver 3.10.12
!source /usr/local/bin/activate
!conda install -q -y --prefix /usr/local python=3.10.12 ujson  # Specify the version here

# Set environment variables for Python and conda paths
!export PYTHONPATH=/usr/local/lib/python3.10/site-packages/
!export CONDA_PREFIX=/usr/local/envs/myenv
```

```
# Python Version in New Virtual Environment (Compatible with ML-Agents)
!python --version
```

3.7 Установка зависимостей

```
# Go inside the repository and install the package (can take 3min)
%cd ml-agents
pip3 install -e ./ml-agents-envs
pip3 install -e ./ml-agents
```

3.8 Загрузка и распаковка архива среды

- Исполняемые файлы нашей среды находятся в ZIP-архиве.
- Нужно скачать архив и поместить его в каталог `./trained-envs-executables/linux/`.

```
mkdir ./trained-envs-executables
mkdir ./trained-envs-executables/linux
```

Мы загружаем файл Huggy.zip из https://github.com/huggingface/Huggy с помощью `wget`.

```
wget "https://github.com/huggingface/Huggy/raw/main/Huggy.zip" -O ./trained-envs-executables/linux/
    Huggy.zip
```

```
%%capture
unzip -d ./trained-envs-executables/linux/ ./trained-envs-executables/linux/Huggy.zip
```

Убедитесь, что файл доступен для запуска.

```
chmod -R 755 ./trained-envs-executables/linux/Huggy
```

3.9 Повторим устройство среды

Пространство состояний: что воспринимает Huggy

Huggy не «видит» окружающую среду. Вместо этого мы предоставляем ему следующую информацию:

- Положение цели (палки)
- Относительное положение Huggy и цели
- Ориентация его лап.

На основе этой информации Huggy **может выбрать следующее действие для достижения своей цели**.



Рис. 11: Huggy: собака-агент в среде Unity ML-Agents [оригинал]

Пространство действий: как может двигаться Huggy

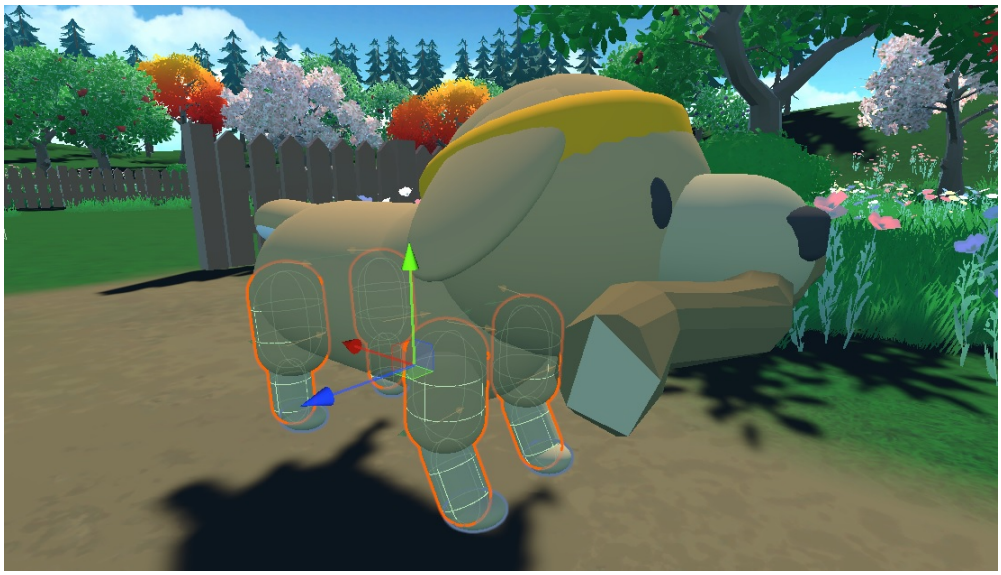


Рис. 12: Управление суставными двигателями лап Huggy [оригинал]

Лапы Huggy приводятся в движение суставными двигателями. Чтобы добраться до цели, Huggy должен **научиться правильно вращать двигатели суставов каждой лапы и таким образом перемещаться**.

Функция вознаграждения

Функция вознаграждения спроектирована так, чтобы **Huggy достигал своей цели**: подбирал палку.

Напомним, что одно из фундаментальных положений обучения с подкреплением — *гипотеза о вознаграждении*: цель можно описать как **максимизацию ожидаемого суммарного вознаграждения**.

В нашем случае Huggy должен **приближаться к палке, не вращаясь слишком сильно**. Поэтому функция вознаграждения должна отражать именно эту цель.

Наша функция вознаграждения:



Рис. 13: Функция вознаграждения для обучения Huggy [оригинал]

- *Бонус за ориентацию*: мы **поощряем Huggy за приближение к цели**.
- *Штраф за время*: постоянный штраф на каждом шаге, который **побуждает его добраться до палки как можно быстрее**.
- *Штраф за вращение*: мы штрафуем Huggy, если **он слишком сильно вращается и слишком резко поворачивает**.
- *Вознаграждение за достижение цели*: поощряем Huggy за то, что **он достиг цели**.

3.10 Проверка конфигурации Huggy

- В ML-Agents **гиперпараметры обучения задаются в файлах config.yaml**.
- В рамках этого ноутбука мы не будем изменять гиперпараметры. Но если вы захотите поэкспериментировать, Unity предоставляет подробную документацию по каждому из них.
- Нужно создать конфигурационный файл для Huggy.
- Перейдите в каталог `/content/ml-agents/config/ppo`.
- Создайте новый файл `Huggy.yaml`.
- Скопируйте и вставьте приведённое ниже содержимое.

```
behaviors:
  Huggy:
    trainer_type: ppo
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
    network_settings:
      normalize: true
      hidden_units: 512
      num_layers: 3
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.995
```

```
        strength: 1.0
    checkpoint_interval: 200000
    keep_checkpoints: 15
    max_steps: 2e6
    time_horizon: 1000
    summary_freq: 50000
```

- Не забудьте сохранить файл!
- **Если вы захотите изменить гиперпараметры**, в ноутбуке Google Colab можно открыть файл config.yaml по пути `/content/ml-agents/config/ppo/Huggy.yaml`.

Теперь всё готово к обучению агента.

3.11 Обучение агента

Чтобы обучить агента, достаточно **запустить mlagents-learn и указать исполняемый файл среды**.

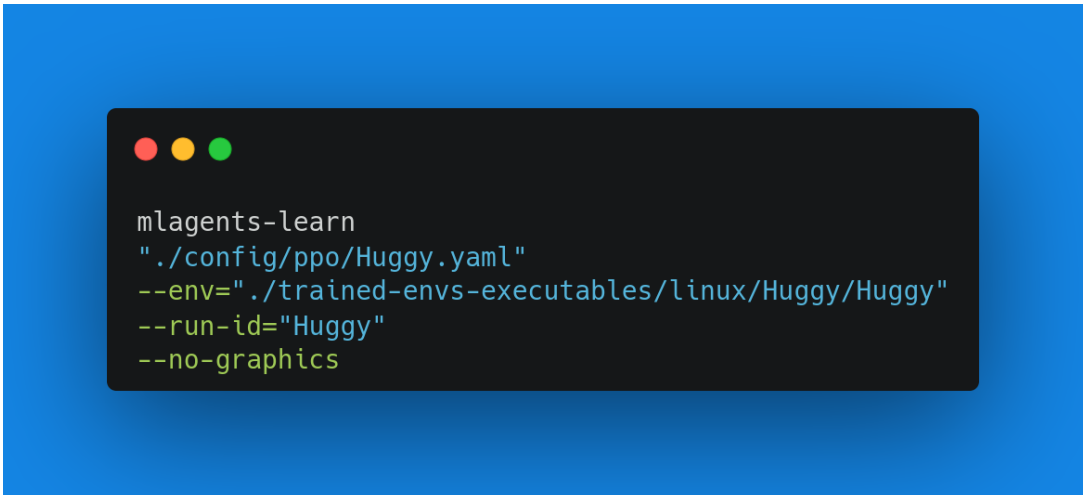


Рис. 14: Структура команды mlagents-learn и её параметры [оригинал]

В ML-Agents обучение запускается отдельным скриптом. Мы задаём четыре параметра:

1. `mlagents-learn <config>`: путь к конфигурационному файлу с гиперпараметрами.
2. `--env`: расположение исполняемого файла среды.
3. `--run-id`: имя запуска обучения.
4. `--no-graphics`: отключение графического отображения во время обучения.

Обучите модель. Если выполнение прервётся, используйте флаг `--resume` для продолжения обучения.

> При первом использовании `--resume` возникнет ошибка; повторно выполните блок, чтобы её обойти.

Обучение займёт от 30 до 45 минут в зависимости от компьютера (не забудьте **настроить GPU**). Можно пока выпить заслуженную чашку кофе.

```
mlagents-learn ./config/ppo/Huggy.yaml --env=./trained-envs-executables/linux/Huggy/Huggy --run-id
    ="Huggy" --no-graphics
```

3.12 Публикация агента на Hugging Face Hub

- Мы обучили агента и теперь **готовы загрузить его на Hub, чтобы играть с Huggy в браузере**.

Чтобы поделиться своей моделью с сообществом, осталось выполнить ещё три шага:

1. Если аккаунта ещё нет, зарегистрируйтесь на Hugging Face: https://huggingface.co/join.
2. Войдите в аккаунт и получите токен доступа на сайте Hugging Face.

- Создайте новый токен (https://huggingface.co/settings/tokens) **с правами записи (write)**.

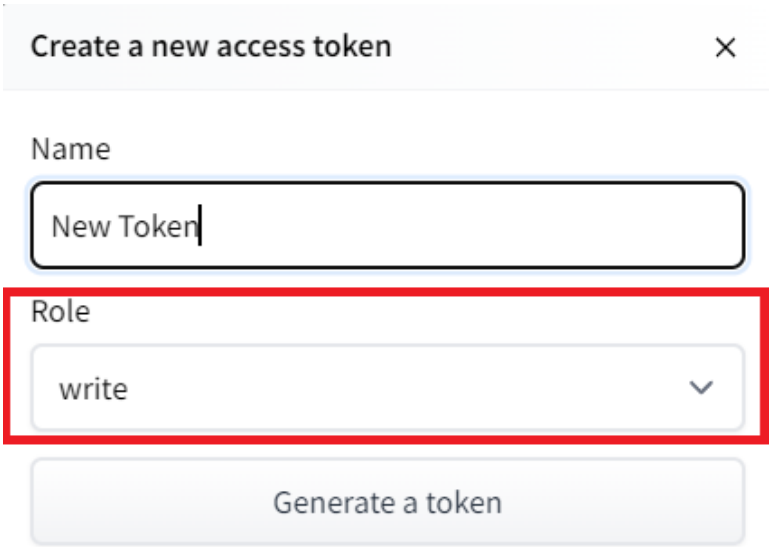


Рис. 15: Создание токена Hugging Face с правом записи [оригинал]

- Скопируйте токен.
- Запустите следующую ячейку и вставьте токен.

```
from huggingface_hub import notebook_login

notebook_login()
```

Если вы не используете Google Colab или Jupyter Notebook, вместо этого выполните команду `huggingface-cli login`.

После этого достаточно запустить `mlagents-push-to-hf`.



Рис. 16: Структура команды mlagents-push-to-hf [оригинал]

Для этой команды указываются четыре параметра:

1. `--run-id`: идентификатор запуска обучения.
2. `--local-dir`: каталог, в котором сохранён агент, то есть `results/<run_id name>`; в моём случае — results/First Training.
3. `--repo-id`: имя репозитория Hugging Face, который нужно создать или обновить. Оно имеет формат `<your huggingface username>/<the repo name>`. Если репозитория ещё нет, **он будет создан автоматически**.
4. `--commit-message`: сообщение коммита, поскольку репозитории Hugging Face используют Git.

```
mlagents-push-to-hf --run-id="HuggyTraining" --local-dir="./results/Huggy" --repo-id="
    ThomasSimonini/ppo-Huggy" --commit-message="Huggy"
```

Если всё прошло успешно, в конце процесса появится примерно такое сообщение (но с другим URL):

```
Your model is pushed to the hub. You can view your model here: https://huggingface.co/
    ThomasSimonini/ppo-Huggy
```

Это ссылка на репозиторий вашей модели. В нём находятся карточка модели с инструкцией по использованию, логи TensorBoard и конфигурационный файл. **Поскольку это Git-репозиторий, вы можете создавать разные коммиты, обновлять модель повторными публикациями, открывать Pull Request и т. д.**

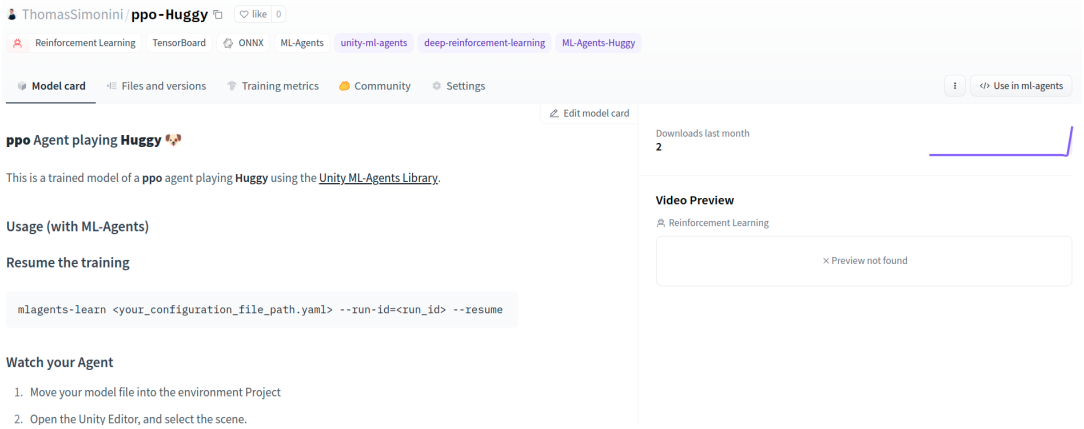


Рис. 17: Пример репозитория обученной модели Huggy и карточки Model Card [оригинал]

А теперь самое интересное: **можно играть с Huggy онлайн**.

3.13 Играем со своим Huggy

Этот шаг самый простой:

- Откройте игру Huggy в браузере: https://huggingface.co/spaces/ThomasSimonini/Huggy.
- Нажмите кнопку Play with my Huggy model.

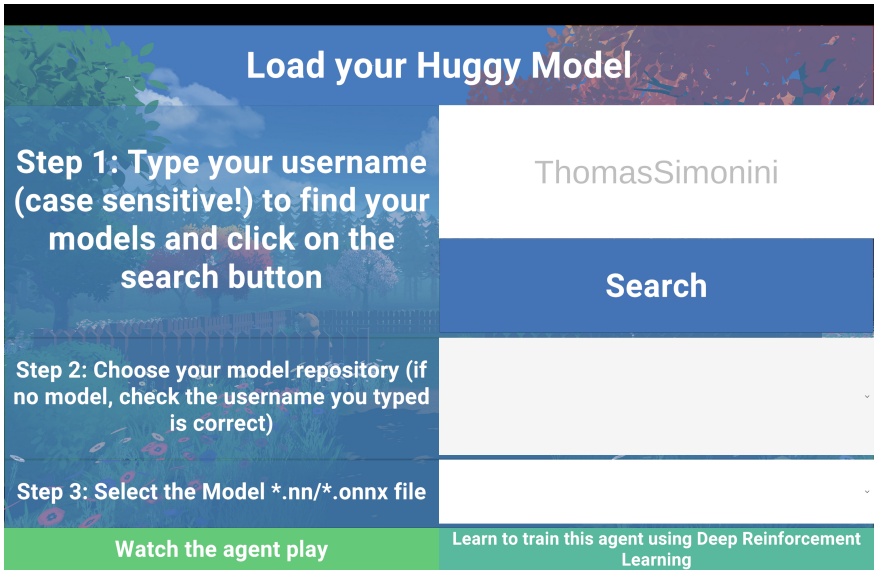


Рис. 18: Браузерная демонстрация Huggy: выбор модели из аккаунта [оригинал]

1. На шаге 1 введите своё имя пользователя (регистр символов важен: например, мой ник — ThomasSimonini, а не thomassimonini или ThOmasImoNInI) и нажмите кнопку поиска.
2. На шаге 2 выберите репозиторий своей модели.
3. На шаге 3 **выберите модель для воспроизведения**:

- У меня их несколько, поскольку мы сохраняли модель каждые 500000 шагов.
- Но для получения самого свежего результата я выбираю `Huggy.onnx`.

Полезно **попробовать модели с разным числом шагов обучения, чтобы увидеть прогресс агента**.

Поздравляем с завершением этого бонусного модуля!

Теперь можно отдохнуть и поиграть со своим Huggy. И **не забудьте поделиться Huggy с друзьями**! Если будете рассказывать о нём в социальных сетях, **отметьте @huggingface и @simoninithomas**.



Рис. 19: Huggy в игровой среде [оригинал]

3.14 Продолжайте учиться и открывать новое

4 Игра с Huggy

Теперь, когда вы обучили Huggy и опубликовали его на Hub, **можно поиграть с ним**.

Здесь всё просто:

- Откройте игру Huggy в браузере: https://huggingface.co/spaces/ThomasSimonini/Huggy.
- Нажмите кнопку Play with my Huggy model.

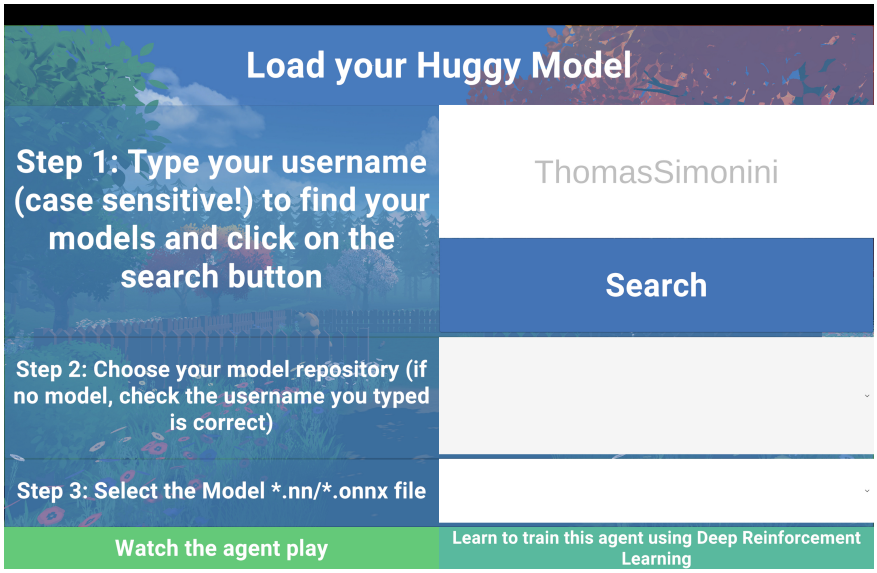


Рис. 20: Браузерная демонстрация Huggy: выбор модели из аккаунта [оригинал]

1. На шаге 1 выберите репозиторий модели — её идентификатор (в моём случае ThomasSimonini/ppo-Huggy).
2. На шаге 2 **выберите модель для воспроизведения**:

- У меня их несколько, поскольку мы сохраняли модель каждые 500000 шагов.

- Если нужна самая свежая версия, я выбираю Huggy.onnx.

Полезно **пробовать разные контрольные точки модели, чтобы наблюдать, как улучшается агент**.

5 Заключение

Поздравляем с завершением этого бонусного модуля!

Теперь можно отдохнуть и поиграть со своим Huggy. И **не забудьте поделиться Huggy с друзьями**! Если будете рассказывать о нём в социальных сетях, **отметьте @huggingface и @simoninithomas**.



Рис. 21: Huggy в игровой среде [оригинал]

Наконец, нам очень важно **узнать ваше мнение о курсе и о том, как его улучшить**. Если у вас есть пожелания, пожалуйста, заполните эту форму.

Продолжайте учиться и открывать новое